

COMPILADORES: PRINCIPIOS, TÉCNICAS Y HERRAMIENTAS

Investigación Sintetizada y Resumen Ejecutivo a 28 Páginas
Capítulo 9: Optimizaciones Independientes de la Máquina

Asignatura:	Compiladores e Intérpretes
Tema:	Optimizaciones Independientes de la Máquina (Capítulo 9 - Aho, Lam, Sethi, Ullman)
Nombre del Estudiante:	_____
Matrícula:	_____
Profesor:	_____
Fecha de Entrega:	20 de Julio de 2026
Formato:	Compendio Estructurado en 28 Páginas

Nota: El presente documento sintetiza la teoría rigurosa del Capítulo 9 y la resolución completa de sus ejercicios de evaluación en una extensión exacta de 28 páginas.

PARTE I. ESTRUCTURA Y RESUMEN DEL CAPÍTULO 9

El presente trabajo condensa la investigación exhaustiva sobre optimizaciones independientes de la máquina, estructurando el contenido exactamente en **veinticinco apartados teóricos centrales** y una **sección integrada de ejercicios resueltos**, ajustados a un formato normativo de 28 páginas.

Organización de la Investigación:

- **Apartados Teóricos (Páginas 3 a 27):** Desarrollo minucioso de cada uno de los 25 temas del Capítulo 9 del libro *Compiladores: principios, técnicas y herramientas* (Aho, Lam, Sethi, Ullman). Cada página abarca de forma autosuficiente un pilar conceptual: análisis de flujo de datos, retículos, dominadores, optimización de ciclos, eliminación de redundancia parcial y análisis simbólico.
- **Compendio de Ejercicios Resueltos (Página 28):** Síntesis formal y rigurosa de las soluciones a las guías prácticas de las secciones 9.1 a 9.8, demostraciones inductivas, tablas de flujo de datos y análisis de reducibilidad.

Objetivos del Proceso de Optimización:

La optimización independiente de la máquina opera sobre la representación intermedia (IR) del código (como el código de tres direcciones y los grafos de flujo de control). Su propósito primordial es transformar el programa para elevar la velocidad de ejecución y reducir el consumo de recursos (memoria, accesos a bus, energía) manteniendo estrictamente inalterada la semántica original.

A lo largo de los apartados se estudia cómo la combinación encadenada de transformaciones locales, globales y de bucles permite alcanzar un nivel superior de eficiencia sin requerir especificaciones particulares del procesador destino.

1. Introducción a las optimizaciones independientes de la máquina

Concepto y Posición en el Compilador

El Capítulo 9 estudia la fase del compilador encargada de transformar el código intermedio (código de tres direcciones, grafos de flujo) de manera independiente a las particularidades del procesador destino. Al trabajar sobre representaciones abstractas antes de la asignación de registros o la selección de instrucciones máquina, las mismas optimizaciones benefician a cualquier arquitectura de hardware.

Naturaleza de la Optimización

Optimizar no implica encontrar el programa óptimo universal, pues en términos generales no existe un algoritmo que garantice producir el código equivalente más rápido para cualquier entrada. En su lugar, el optimizador aplica transformaciones bien fundamentadas que ofrecen mejoras sustanciales en programas reales sin disparar de forma excesiva el tiempo de compilación.

Ejes Principales

Tres pilares sostienen la disciplina: 1) Reconocimiento de redundancias (cálculos repetidos y variables muertas); 2) Recolección global de información a través del análisis de flujo de datos en todos los caminos posibles; 3) Análisis de la estructura de ciclos, donde reducciones menores de costo se multiplican exponencialmente al ejecutarse repetidamente.

Conclusión Teórico-Práctica

Esta etapa conecta la matemática discreta (conjuntos, retículos, grafos de control) con la práctica del software para ejecutar menos instrucciones y evitar accesos innecesarios a memoria.

2. Objetivos y criterios de una optimización correcta

Criterio Fundamental: Preservación de la Semántica

El criterio supremo e inflexible de toda optimización es la preservación del significado observable del programa. El código transformado debe arrojar exactamente los mismos resultados y conservar los efectos laterales válidos para todas las entradas admisibles por la especificación del lenguaje.

Criterio de Beneficio Real

Toda transformación debe ofrecer una ganancia justificable en tiempo de ejecución, tamaño del ejecutable o eficiencia energética. Mover una operación fuera de un ciclo iterativo aporta un beneficio elevado, mientras que la misma mejora en un bloque lineal rara vez justifica la complejidad del análisis.

Costo de Compilación y Complejidad de Ingeniería

El tiempo y espacio requeridos por el compilador deben ser razonables. Análisis sumamente detallados (como el análisis preciso de alias) pueden ralentizar desmedidamente la compilación. Se busca un equilibrio de ingeniería entre la precisión del análisis y la mantenibilidad del código del compilador.

Interacción Encadenada de Pasadas

Las optimizaciones interactúan cooperativamente: la eliminación de subexpresiones prepara el código para la reducción en fuerza; la propagación de copias deja asignaciones muertas que luego se borran. Por ello se aplican múltiples pasadas de optimización.

3. Fuentes de redundancia en el código intermedio

Origen en la Traducción Automática

La traducción directa desde lenguajes de alto nivel a representación intermedia genera abundantes operaciones temporales. El acceso a arreglos multidimensionales desglosa multiplicaciones para calcular desplazamientos y sumas de direcciones, repitiendo cálculos si se accede varias veces a posiciones contiguas.

Estilo del Programador y Claridad del Código

Los programadores suelen introducir variables intermedias y evaluar condiciones repetidas para mantener la modularidad y legibilidad. El optimizador no altera este estilo claro en el código fuente, sino que elimina el trabajo redundante internamente en la representación intermedia.

Ciclos como Fuente Principal de Redundancia

Dentro de los bucles, las expresiones invariantes se evalúan repetidamente en cada iteración en traducciones ingenuas. Reemplazar multiplicaciones de índices por sumas incrementales constantes reduce drásticamente el costo computacional.

Redundancia Secundaria por Transformaciones Previas

Una optimización a menudo descubre nuevas redundancias. Reemplazar una variable por una constante puede volver inalcanzable una rama de salto, lo que a su vez elimina dependencias y vuelve muertas a otras asignaciones.

4. El ejemplo de Quicksort como caso de estudio

Contexto del Algoritmo Quicksort

El texto analiza el bucle interno de Quicksort, donde dos índices recorren un arreglo desde extremos opuestos comparando elementos contra un pivote. Las referencias a arreglos como `a[i]` generan subexpresiones como `4*i` (asumiendo elementos de 4 bytes).

Paso 1: Eliminación de Subexpresiones Comunes

Si el desplazamiento `4*i` o el valor de `a[i]` ya fueron calculados previamente y no ha habido escrituras en la memoria intermitente, el compilador reutiliza los valores guardados en temporales, eliminando multiplicaciones y cargas redundantemente repetidas.

Paso 2: Reducción en Fuerza e Inducción

En lugar de multiplicar `4*i` en cada vuelta, se crea una variable temporal que inicia en el valor base y se incrementa en 4 unidades por iteración. Las multiplicaciones pesadas dentro del ciclo se transforman en adiciones ligeras.

Paso 3: Limpieza e Integración de Código Muerto

Al expresar las comparaciones del ciclo directamente en términos del desplazamiento temporal (ej. `p <= 792`), el índice `i` original deja de usarse. La eliminación de código muerto remueve los incrementos e inicializaciones de `i`, obteniendo un bucle óptimo.

5. Transformaciones que preservan la semántica

Transformaciones Locales vs. Globales

Las transformaciones locales operan de forma aislada dentro de un bloque básico (secuencia lineal de instrucciones sin saltos). Las transformaciones globales abarcan múltiples bloques a lo largo del grafo de flujo de control (CFG) y exigen conocer el flujo de datos.

Desafío de los Efectos Laterales

La validez de una transformación depende de garantizar la ausencia de efectos laterales no deseados. Las llamadas a procedimientos pueden modificar memoria global, las escrituras por punteros pueden alterar variables indirectas y las excepciones pueden hacer visible el orden exacto de las operaciones.

Análisis de Alias y Reglas del Lenguaje

Un compilador seguro debe considerar qué variables pueden compartir la misma dirección de memoria (alias). Si no se puede probar plenamente la independencia de dos accesos, se asume una posible interferencia para evitar transformaciones erróneas.

Principio Conservador de Diseño

Ante cualquier incertidumbre en el análisis, el compilador debe conservar el código original. La exactitud semántica del ejecutable prevalece incondicionalmente sobre cualquier ganancia potencial de velocidad.

6. Eliminación de subexpresiones comunes globales

Definición de Subexpresión Común Global

Una expresión e (como $x + y$) es una subexpresión común global en un punto p si e se calculó en un bloque anterior y ninguno de sus operandos (x o y) ha sido redefinido a lo largo de ningún camino desde aquel cálculo hasta p .

Requisito de Disponibilidad Global

Para reemplazar la evaluación de e en p por el valor previamente calculado, la expresión debe estar disponible por todos los caminos posibles de entrada. Si existe un solo camino donde los operandos se modifican o no se calculan, la sustitución no es segura.

Uso de Variables Temporales Auxiliares

Si el resultado original se almacenó en una variable que luego se redefine, el compilador asigna un temporal t para guardar e en su primera evaluación. Los usos posteriores de e se reemplazan por la lectura directa de t .

Balance con la Presión sobre Registros

Aunque reduce operaciones aritméticas, mantener variables temporales vivas durante más tiempo incrementa el uso de registros físicos. El compilador pondera este costo frente a la complejidad de las operaciones eliminadas.

7. Propagación de copias y de constantes

Propagación de Copias

Tras una asignación $x = y$, sustituir los usos posteriores de x por y permite eliminar nombres temporales redundantes. Si todas las referencias a x son reemplazadas y x no vuelve a ser leída, la instrucción $x = y$ se elimina como código muerto.

Propagación y Plegado de Constantes

Si una asignación $x = c$ establece una constante conocida, los usos de x se sustituyen por c . Si los operandos de una expresión son constantes conocidas (ej. $3 + 5$), el compilador calcula el resultado 8 en tiempo de compilación (fusión o plegado de constantes).

Impacto de Bifurcaciones y Ciclos

En presencia de ciclos o puntos de reunión, una variable puede recibir distintas definiciones según el camino tomado. La propagación global requiere verificar que una única definición alcance la instrucción objetivo por todas las rutas de control.

Importancia del Análisis de Flujo de Datos

Determinar la validez de propagar una constante exige resolver ecuaciones globales de alcance. El análisis no puede limitarse a la estructura visual del programa, sino a la totalidad de los caminos posibles de ejecución.

8. Eliminación de código muerto e inalcanzable

Concepto de Código Muerto (Dead Code)

Código muerto se refiere a instrucciones computacionalmente válidas cuyo valor resultante nunca es consumido en ninguna ejecución posterior del programa, o a asignaciones sobrescritas antes de ser leídas.

Detección mediante Variables Vivas

A través del análisis de liveness (variables vivas), se determina si el valor guardado en una variable será leído en el futuro. Si la variable de destino no está viva a la salida de la instrucción y la operación no produce efectos laterales, se elimina.

Cuidado con Funciones y Efectos Secundarios

Incluso si el valor retornado por una función no se utiliza, la llamada no puede eliminarse si la función realiza operaciones de E/S, modifica variables globales o interactúa con el sistema operativo.

Código Inalcanzable (Unreachable Code)

A diferencia del código muerto, el código inalcanzable corresponde a bloques a los que nunca llega el flujo de control (tras saltos incondicionales o condiciones falsas permanentes). Su eliminación reduce el tamaño del binario y acelera otros análisis.

9. Movimiento de código invariante de ciclo

Expresiones Invariantes en Ciclos

Una expresión es invariante respecto a un bucle si sus operandos son constantes o se definen exclusivamente fuera del bucle, o si dependen de otras cantidades que también son invariantes de ciclo.

Ubicación en el Preencabezado (Preheader)

Para evitar evaluar una expresión invariante en cada vuelta, el compilador crea un bloque llamado preencabezado justo antes de la entrada del ciclo y traslada la evaluación de la expresión a dicho bloque.

Condiciones de Seguridad de Dominación

Mover una instrucción es seguro solo si el bloque donde originalmente se encuentra domina a todas las salidas del bucle o si la expresión no puede provocar excepciones (como división entre cero o violación de memoria).

Beneficio Computacional

El movimiento de invariantes ahorra tiempo apreciable en bucles anidados intensivos, disminuyendo el número de cargas a memoria y operaciones aritméticas redundantes dentro del cuerpo del ciclo.

10. Variables de inducción y reducción en fuerza

Variables de Inducción Básicas y Derivadas

Una variable i es de inducción básica si sus cambios dentro del ciclo son incrementos constantes ($i = i + c$). Una variable j es derivada si su valor es una función lineal de i (ej. $j = c1 * i + c2$).

Mecanismo de Reducción en Fuerza

Consiste en reemplazar operaciones aritméticas costosas (como multiplicaciones) por operaciones más ligeras (adiciones). Si j se calcula como $4*i$ en cada vuelta, se crea un temporal $t = 4*i$ inicial y se incrementa en 4 dentro del bucle.

Eliminación de Variables de Inducción

Una vez que las variables derivadas han sido transformadas en temporales incrementales, la variable de inducción básica original (como i) puede volverse muerta si las condiciones de parada del bucle se reexpresan en función del temporal.

Optimización en bucles anidados

En estructuras de ciclos anidados, esta técnica permite encadenar optimizaciones de punteros y desplazamientos de arreglos, transformando bucles pesados en recorridos secuenciales optimizados.

11. Introducción al análisis de flujo de datos

Definición de Análisis de Flujo de Datos

Es la técnica formal mediante la cual el compilador recopila información abstracta sobre los valores posibles de las variables en cada punto del programa, estructurando el código como un grafo de flujo de control (CFG).

Hechos Abstractos y Puntos de Programa

Cada problema de análisis define un tipo de hecho: qué asignaciones pueden alcanzar un punto, qué variables están vivas o qué expresiones se encuentran disponibles. Se asocia un estado de información a la entrada (ENT) y salida (SAL) de cada bloque básico.

Dirección del Flujo (Hacia Adelante vs. Hacia Atrás)

Los problemas fluyen según la dirección de ejecución. Las definiciones de alcance y expresiones disponibles se propagan hacia adelante. El análisis de variables vivas fluye hacia atrás, desde los usos futuros hacia las definiciones anteriores.

Aproximación Conservadora (Seguridad)

Al existir múltiples caminos posibles en bifurcaciones, el compilador aplica aproximaciones conservadoras: prefiere asumir un escenario posible adicional antes que omitir un comportamiento real que arruine la semántica.

12. Esquemas y ecuaciones de flujo de datos

Estructura de las Ecuaciones de Flujo

Para cada bloque B, la información de salida SAL[B] se relaciona con la entrada ENT[B] mediante una función de transferencia fB. En análisis hacia adelante: $SAL[B] = fB(ENT[B])$. En análisis hacia atrás: $ENT[B] = fB(SAL[B])$.

Operadores de Reunión (Confluencia)

En puntos donde convergen varios caminos, la información de los predecesores se combina mediante el operador de reunión. En análisis de posibilidad (may-analysis) se usa la Unión (U); en análisis de certeza (must-analysis) la Intersección ($\cap$).

Esquema Gen-Eliminar (Gen-Kill)

Muchos marcos utilizan las variables gen[B] (hechos generados dentro del bloque) y kill[B] (hechos destruidos por el bloque). La ecuación fundamental es: $SAL[B] = gen[B] \cup (ENT[B] - kill[B])$.

Condiciones de Frontera e Inicialización

Se definen valores para la entrada global del programa (ENTRY) o salida (EXIT). Los bloques internos se inicializan con conjuntos vacíos o el conjunto universal U para asegurar convergencia conservadora.

13. Definiciones de alcance (Reaching Definitions)

Concepto de Definición de Alcance

Una asignación $d: x = v$ alcanza un punto p si existe al menos un camino de control desde d hasta p a lo largo del cual x no es redefinida por otra instrucción.

Construcción de los Conjuntos Gen y Kill

Para un bloque B , $gen[B]$ contiene la última definición de cada variable asignada en B . $kill[B]$ contiene todas las demás definiciones de esas mismas variables existentes en otros bloques del programa.

Ecuaciones de Flujo y Dirección

El análisis es hacia adelante con operador de reunión Unión ($\cup$): $ENT[B] = \cup_{P \text{ predecesor}} SAL[P]$ y $SAL[B] = gen[B] \cup (ENT[B] - kill[B])$. El proceso inicia con conjuntos vacíos.

Aplicaciones Directas

Permite construir las cadenas Def-Uso (uso de variables vinculado a sus definiciones). Si a un uso le alcanza una única definición que asigna una constante, se autoriza la propagación de constantes.

14. Análisis de variables vivas (Live Variables)

Concepto de Variable Viva (Liveness)

Una variable x está viva en un punto p si existe algún camino en el CFG que parte de p a lo largo del cual el valor de x es leído antes de sufrir cualquier nueva redefinición.

Conjuntos Locales: Uso y Def

Para cada bloque B , $uso[B]$ es el conjunto de variables leídas en B antes de ser redefinidas. $def[B]$ es el conjunto de variables asignadas en B antes de ser leídas.

Dirección del Flujo y Ecuaciones

Fluye hacia atrás con operador de reunión Unión ($\cup$): $SAL[B] = \cup_{S \text{ sucesor}} ENT[S]$ y la función de transferencia es $ENT[B] = uso[B] \cup (SAL[B] - def[B])$.

Aplicación en Código Muerto y Registros

Si una asignación modifica a x pero x no pertenece a $SAL[B]$, la asignación es muerta y puede borrarse. Además, dos variables que no están vivas simultáneamente pueden compartir el mismo registro de máquina.

15. Expresiones disponibles (Available Expressions)

Concepto de Expresión Disponible

Una expresión $x + y$ está disponible en un punto p si en TODOS los caminos de control desde la entrada hasta p se ha evaluado $x + y$ sin que x o y hayan sido redefinidos con posterioridad.

Conjuntos e_gen y e_kill

$e_gen[B]$ contiene las expresiones calculadas en B cuyos operandos no son redefinidos posteriormente en B . $e_kill[B]$ contiene todas las expresiones del programa que utilicen variables modificadas en B .

Dirección y Operador de Intersección

Fluye hacia adelante utilizando la Intersección ($\cap$) como operador de reunión: $ENT[B] = \cap_{P \text{ predecesor}} SAL[P]$. La salida es $SAL[B] = e_gen[B] \cup (ENT[B] - e_kill[B])$.

Inicialización del Sistema

Para garantizar certeza, los bloques internos se inicializan con el conjunto universal U de expresiones del programa, mientras que $ENT[ENTRY]$ se fija como conjunto vacío.

16. Semirretículos y relación de orden

Fundamentación Matemática

Los problemas de flujo de datos se formalizan sobre un semirretículo inferior $(L, \wedge)$, constituido por un conjunto de valores L y una operación de reunión $\wedge$ asociativa, conmutativa e idempotente ($a \wedge a = a$).

Relación de Orden Parcial Induced

La operación de reunión define una relación de orden parcial: $a \leq b$ si y solo si $a \wedge b = a$. En análisis de certeza con intersección, el orden coincide con la inclusión de conjuntos.

Altura del Semirretículo

La altura de un semirretículo es la longitud máxima de una cadena estrictamente descendente. Si la altura es finita y las funciones son monótonas, los algoritmos de punto fijo garantizan terminación.

Productos de Semirretículos

Permiten analizar múltiples variables de forma concurrente, componiendo elementos abstractos (como vectores de estados de constantes) en marcos unificados.

17. Funciones de transferencia, monotonía y distributividad

Propiedades de las Funciones de Transferencia

La función f_B representa el cambio del estado abstracto al atravesar un bloque. Para una secuencia de instrucciones, la función del bloque se obtiene mediante la composición secuencial de funciones individuales.

Monotonía

Una función f es monótona si conserva el orden del semirretículo: $x \leq y$ implica $f(x) \leq f(y)$. La monotonía asegura que el proceso iterativo avance de forma estable sin oscilaciones.

Distributividad

Una función es distributiva si $f(x \wedge y) = f(x) \wedge f(y)$. La distributividad garantiza que la solución iterativa converja exactamente al resultado ideal de reunir los datos sobre todos los caminos reales (MOP).

Implicación en Marcos Prácticos

Los problemas gen-eliminar estándar son distributivos. Problemas más complejos (como la propagación de constantes con alias) son monótonos pero no distributivos, obteniendo soluciones seguras pero conservadoras.

18. Algoritmo iterativo y punto fijo

Mecanismo del Algoritmo Iterativo

El algoritmo evalúa continuamente las ecuaciones de flujo recalculando ENT y SAL para cada bloque básico hasta que en una pasada completa ningún valor experimente cambios.

Concepto de Punto Fijo (Fixpoint)

El estado final donde las ecuaciones se satisfacen simultáneamente se denomina punto fijo. Dado que el semirretículo es de altura finita y las funciones son monótonas, la convergencia matemática está garantizada.

Algoritmo de Lista de Trabajo (Worklist Algorithm)

En lugar de evaluar todos los bloques en cada iteración, se mantiene una lista de trabajo con los bloques cuyos insumos han cambiado. Esto optimiza drásticamente el tiempo de compilación.

Orden de Visita Óptimo

Reordenar los bloques según el flujo de control (postorden inverso en análisis hacia adelante) reduce el número de pasadas necesarias para alcanzar la estabilización.

19. Significado de las soluciones MOP y MFP

Solución MOP (Meet-Over-All-Paths)

La solución MOP representa el valor de flujo ideal que se obtendría al calcular la función de transferencia a lo largo de cada camino individual desde la entrada y luego reunir todos los resultados finales.

Solución MFP (Maximum Fixpoint)

Es la solución que calcula el algoritmo iterativo estándar, acumulando y reuniendo información en cada nodo de confluencia antes de proseguir por los bloques siguientes.

Teorema de Coincidencia de Kildall

Si el marco de flujo de datos es distributivo, la solución del algoritmo iterativo (MFP) es exactamente igual a la solución ideal sobre todos los caminos ($MOP = MFP$).

Pérdida de Precision en Marcos No Distributivos

Si las funciones no son distributivas, MFP es un subconjunto conservador de MOP ($MFP \leq MOP$). Ofrece total seguridad semántica, aunque puede perder cierta precisión de análisis.

20. Propagación de constantes

Dominio Abstracto de Constantes

Para cada variable se define un retículo con tres niveles: Top (indeterminado / sin inicializar), valores constantes c , y Bottom (no constante / valor variable desconocido).

Operador de Reunión en la Propagación

$\text{Top} \wedge v = v$; $c_1 \wedge c_2 = c_1$ si $c_1 == c_2$; $c_1 \wedge c_2 = \text{Bottom}$ si $c_1 != c_2$; $\text{Bottom} \wedge v = \text{Bottom}$. Si dos caminos convergen con constantes distintas, la variable se vuelve Bottom.

Poda de Ramas y Simplificación de Flujo

Si la condición de un salto se evalúa a una constante conocida (ej. `if (1 == 1)`), el compilador elimina la arista correspondiente al camino falso, descubriendo código inalcanzable.

Detección de Variables No Inicializadas

El marco abstracto puede extenderse para emitir advertencias de compilación si una variable se lee en un estado con componentes Top sin asignar previamente.

21. Eliminación de redundancia parcial

Concepto de Redundancia Parcial (PRE)

Una expresión es parcialmente redundante si se calcula a lo largo de algunos caminos que llegan a un punto, pero no por todos. La eliminación de subexpresiones comunes estándar no puede actuar.

Estrategia de Transformación

Se insertan evaluaciones de la expresión en los caminos donde faltaba, convirtiendo la redundancia parcial en redundancia total para luego eliminar la evaluación original.

Algoritmo de Movimiento de Código Diferido (Lazy Code Motion)

Resuelve cuatro análisis de flujo para colocar las expresiones en posiciones que sean totalmente seguras y lo más tardías posible, evitando extender innecesariamente la vida de los temporales.

Beneficios Integrados

Lazy Code Motion unifica en un solo algoritmo la eliminación de subexpresiones comunes globales y el movimiento de código invariante de ciclos.

22. Dominadores y árbol de dominadores

Relación de Dominación

Un nodo d domina a un nodo n (escrito $d \text{ dom } n$) si todo camino en el CFG desde el nodo de entrada hasta n pasa obligatoriamente por el nodo d .

Dominador Inmediato y Árbol de Dominadores

Todo nodo n (salvo ENTRY) posee un único dominador estricto más cercano llamado dominador inmediato ($\text{idom}(n)$). El conjunto de estas relaciones forma un árbol jerárquico.

Detección de Aristas Posteriores (Back Edges)

Una arista $n \rightarrow h$ es una arista posterior si el nodo cabeza h domina al nodo cola n . La presencia de aristas posteriores identifica la estructura de los ciclos en el programa.

Aplicación en Estructuración de Bucles

El árbol de dominadores permite determinar puntos seguros para la inserción de preencabezados y validar el movimiento invariante de código.

23. Búsqueda en profundidad, reducibilidad y ciclos naturales

Recorrido DFS y Clasificación de Aristas

Un recorrido DFS genera un árbol de expansión. Las aristas se clasifican en: de árbol, de avance, de retirada y de cruce. Las de retirada van hacia antecesoros en el árbol DFS.

Grafos de Flujo Reducibles

Un CFG es reducible si todas sus aristas de retirada son también aristas posteriores de dominación. Esto equivale a que sus ciclos tengan una única entrada de control.

Ciclos Naturales

El ciclo natural de una arista posterior $n \rightarrow h$ es el conjunto de nodos h junto a todos los nodos que pueden alcanzar a n sin pasar por h . El nodo h actúa como encabezado único.

Garantía de Estructura

Los programas escritos con estructuras de control disciplinadas (while, for, if) producen siempre grafos reducibles, optimizando el rendimiento de los algoritmos iterativos.

24. Análisis basado en regiones

Enfoque Jerárquico por Regiones

Divide el CFG en porciones contiguas llamadas regiones. Una región posee un nodo de entrada único que domina a todos los nodos internos de la región.

Operaciones de Reducción T1 y T2

T1 elimina un autociclo $n \rightarrow n$ en un nodo. T2 combina un nodo n con su único predecesor m . Un grafo es reducible si y solo si puede reducirse a un único nodo mediante T1 y T2.

Construcción de Funciones de Transferencia Compuestas

Se derivan ecuaciones de flujo para bloques básicos, luego para regiones compuestas y ciclos, hasta obtener la función global del procedimiento de forma analítica.

División de Nodos para Grafos No Reducibles

En grafos no reducibles (con múltiples entradas a un ciclo), se duplican nodos para desglosar entradas independientes y convertirlos en grafos reducibles procesables.

25. Análisis simbólico y conclusión del capítulo

Concepto de Análisis Simbólico

Representa los valores de las variables como funciones simbólicas de las variables de entrada iniciales, en lugar de aproximaciones constantes estáticas.

Expresiones Afines y Relaciones en Ciclos

Permite derivar relaciones como $y = x - 1$ y $z = x - 2$, demostrando simbólicamente que dos índices de arreglos nunca colisionan o que un bucle se ejecuta un número k determinado de veces.

Cierre en Estructura de Bucles

Integrado con el análisis por regiones, el análisis simbólico construye formas cerradas para inducciones, permitiendo vectorizar o paralelizar bucles automáticamente.

Conclusión del Capítulo 9

Las optimizaciones independientes de la máquina constituyen el núcleo formal del compilador. La combinación de teoría de flujo de datos, retículos, dominadores y transformaciones encadenadas garantiza programas altamente eficientes con rigurosa seguridad semántica.

PARTE II. COMPENDIO DE EJERCICIOS RESUELTOS (SECCIONES 9.1 A 9.8)

Síntesis rigurosa y soluciones analíticas consolidadas de las guías prácticas del Capítulo 9.

Sección 9.1: Quicksort y Transformaciones Fundamentales

- **Ejercicio 9.1.1 (Análisis de bucles en Fig 9.10):** a) Identificación de ciclo interno {B3, B4} (cabeza B3 por B4->B3) y ciclo externo {B2, B3, B4, B5} (cabeza B2 por B5->B2). b) La definición $a=1$ en B1 alcanza las líneas (3),(4),(6),(8),(9) permitiendo propagar 1. c) En el ciclo interno, $a+b$ en (6) es redundante respecto a (3). d) La variable e es de inducción básica ($e=e+1$); b es básica en el ciclo externo ($b=b+1$) y c es derivada ($c=b+1$). e) $a+b$ es invariante en el ciclo interno y se mueve al preencabezado.
- **Ejercicios 9.1.2 - 9.1.4:** Se reemplazan multiplicaciones $t1=10*i$ y $t6=88*(i-1)$ por temporales p con incrementos $p=p+8$ y $p=p+88$. En multiplicación de matrices (9.1.3a) y Criba de Eratóstenes (9.1.3b), se extraen desplazamientos de filas e incrementos de punteros fuera de bucles internos, eliminando variables de índice secundarias.

Sección 9.2: Tablas de Flujo de Datos (Definiciones, Vivas, Disponibles)

- **Ejercicios 9.2.1 - 9.2.3:** Construcción de tablas ENT/SAL. En 9.2.1 (Definiciones de alcance) B1 produce {d1,d2} y elimina {d8,d10,d11}. En 9.2.2 (Expresiones disponibles) e-gen de B2 es {a+b, c-a} con intersección universal. En 9.2.3 (Variables vivas) el flujo backward obtiene $SAL(B1)=\{a,b,e\}$ y $ENT(B6)=\{b,d\}$.
- **Ejercicios 9.2.5 - 9.2.10:** Demostración por inducción en n instrucciones para la fórmula $genB/eliminarB$ (9.2.5). Prueba de convergencia monótona de la iteración (9.2.6). La inicialización de expresiones disponibles con U es obligatoria para no perder disponibilidad en bucles (9.2.10).

Sección 9.3: Retículos, Monotonía y Soluciones MOP vs MFP

- **Ejercicios 9.3.1 - 9.3.5:** El producto de retículos de 3 componentes forma un cubo booleano $2^3=8$ (9.3.1). fS en 9.3.3 no es monótona ($fS(1)=2$, $fS(2)=1$), provocando oscilación infinita sin convergencia. Demostración por inducción de $X_i \leq MOP_i$ (9.3.4). Demostración de distributividad en marcos gen -eliminar: $f(X \cup Y) = f(X) \cup f(Y)$ (9.3.5).

Sección 9.4 - 9.5: Propagación Avanzada, PRE y Lazy Code Motion

- **Ejercicios 9.4.1 - 9.5.3:** Para detectar variables sin inicializar se utiliza un par (valor, estado_init) con reunión OR (9.4.1). En 9.5.1/9.5.2 (Lazy Code Motion) se colocan expresiones anticipadas en las posiciones más tardías seguras (Primeras/Últimas), reduciendo la vida de temporales. 9.5.3 presenta el algoritmo dual para código parcialmente muerto empujando asignaciones hacia adelante.

Sección 9.6 - 9.8: Dominadores, Reducibilidad, Regiones y Simbólico

- **Ejercicios 9.6.1 - 9.6.13:** Árbol de dominadores en cadena $B1 \rightarrow B2 \rightarrow B3$ con hijos $B4, B5$ (9.6.1). Caracterización de reducibilidad: un CFG es reducible ssi al quitar aristas posteriores queda un DAG acíclico (9.6.5). En 9.7.1 - 9.7.6, las transformaciones T1 (autociclos) y T2 (predecesor único) reducen grafos estructurados a un nodo. En 9.8.1 - 9.8.3 se derivan mapas simbólicos $f^i(m)$ produciendo formas cerradas $e_j = m(b) + (j-1)*m(a)$ para iteraciones de bucles.